

Vuông = Nhiều đáp án

Tròn = 1 đáp án

Lecture 4: Use Case Analysis

Câu 1: Phương pháp phân tích thiết kế hướng đối tượng được thực hiện theo hướng:

- ☐ Top-down
- ☐ Bottom-up
- ☒ Xuất phát từ Middle (middle-up or middle-down)

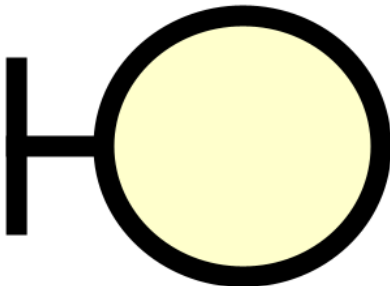
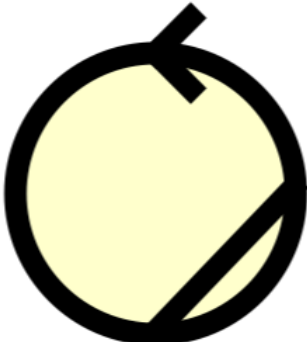
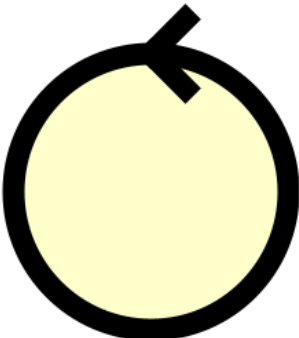
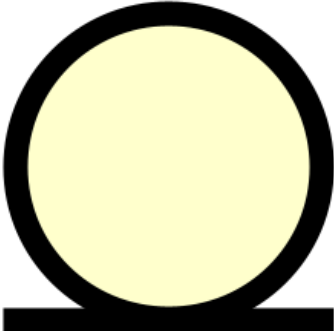
Câu 2: Trong mô hình BCE, các lớp phân tích bao gồm những loại nào sau đây?

- ☒ Entity Classes
- ☐ Model Classes
- ☒ Boundary Classes
- ☐ Service Classes
- ☒ Control Classes
- ☐ Middleware Classes
- ☐ Validation Classes

Câu 3: Boundary Classes có thể bao gồm những loại nào sau đây?

- ☒ User Interface Classes
- ☒ System Interface Classes
- ☒ Device Interface Classes (Hardware Interface)

Câu 4: Để phân biệt một lớp **mô hình hóa dữ liệu** trong giai đoạn phân tích của UML (Analysis Model), stereotype nào sau đây là phù hợp nhất để gán cho lớp đó?

	
☐ Tùy chọn 1 (Boundary)	☐ Tùy chọn 2
	
☐ Tùy chọn 3 (Controller)	☒ Tùy chọn 4 (Entity)

5. Trong mô hình BCE, lớp nào chịu trách nhiệm quản lý luồng nghiệp vụ (business logic) và điều phối các lớp khác để hoàn thành một Use Case?

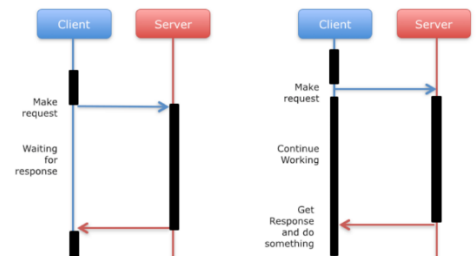
- «entity» (mô hình hoá dữ liệu)
- «boundary» (nhận tương tác)
- «control» (điều phối)

6. Trong giai đoạn phân tích use case, việc định danh các lớp phân tích (Boundary Class, Control Class, Entity Class) cần những nguồn thông tin nào?

- ☒ Dựa vào Use Case và các flows/tính năng được mô tả (flow of events, scenarios).
- ☐ Dựa vào các loại biểu đồ như Sequence Diagram, Communication Diagram. (xác định BCE mới vẽ được seq, commu)
- ☐ Dựa vào tài liệu đặc tả (Use Case Specification, Supplementary Specs, outline của use case).
- ☐ Dựa vào mong muốn khách hàng chưa được đặc tả/thẩm định thành yêu cầu chính thức của phần mềm. (SRS)

7. Cho các mệnh đề sau về truyền thông điệp Synchronous (đồng bộ) vs. Asynchronous (bất đồng bộ):

- (1) Synchronous: Khi gửi một thông điệp, Client phải chờ cho đến khi thông điệp đó hoàn tất.
- (2) Asynchronous: Khi gửi một thông điệp, Client có thể tiếp tục xử lý các công việc khác, không phải chờ phản hồi ngay.
- (3) Asynchronous giúp tăng tính responsive và giảm ràng buộc theo thời gian (temporal coupling).
- (4) Asynchronous thường khó debug hơn. (không có trong slide nhưng đúng trong thực tế, tự suy luận)



Chọn đáp án đúng:

- (1), (2), (3) đúng
- ☒ (1), (2), (3), (4) đều đúng
- Chỉ (2) và (4) đúng
- Chỉ (1) và (2) đúng

8. Biểu đồ tương tác (interaction diagram) có thể bao gồm các loại biểu đồ nào sau đây?

- ☐ Use Case Diagram
- ☒ Sequence Diagram
- ☐ Activity Diagram (biểu đồ nghiệp vụ) từ UC -> 1 diagram chứa nhiều UC để hoàn thành 1 nghiệp vụ
- ☒ Communication Diagram (seq ~ commu)
- ☐ Class Diagram
- ☐ Deployment Diagram
- ☐ Component Diagram

9. Trong Interaction Diagram, ta thấy một đối tượng của Boundary Class **B1** truyền thông điệp **m1()** tới một đối tượng của Control Class **C1**. Khi chuyển Interaction Diagram này sang Analysis Class Diagram tương ứng, khẳng định nào sau đây là đúng?

- Trong Analysis Class Diagram, Lớp **B1** chứa phương thức **m1()**.
- ☒ Trong Analysis Class Diagram, Lớp **C1** chứa phương thức **m1()**. (truyền vào thằng nào thì thằng đấy chứa)

- Trong Analysis Class Diagram, không lớp nào chứa phương thức **m1()** vì **m1()** chỉ là thông điệp trong Sequence Diagram.

10. Hãy chọn các phát biểu đúng về quá trình mô hình hóa dữ liệu (Data Modelling).

Các thuật ngữ được sử dụng:

- Mô hình hóa dữ liệu trừu tượng: Conceptual Data Modelling
- Mô hình hóa dữ liệu logic: Logical Data Modelling
- Mô hình hóa dữ liệu vật lý: Physical Data Modelling (phụ thuộc DBMS)
- Ở bước mô hình hóa dữ liệu trừu tượng: Đầu vào là các lớp thực thể (entity class diagram), Đầu ra là sơ đồ thực thể liên kết (entity relationship diagram). (Từ seq -> analysis class mà mỗi class là các thành phần trong seq -> mapping các thành phần (thực thể, thuộc tính) vào ERD)
- Bước mô hình hóa dữ liệu trừu tượng phụ thuộc vào việc chọn hệ quản trị cơ sở dữ liệu (DBMS) nào. (ERD là DBMS independent)
- Ở bước mô hình hóa dữ liệu vật lý, việc chọn cơ sở dữ liệu quan hệ hay phi quan hệ sẽ quyết định hệ quản trị cơ sở dữ liệu sẽ được sử dụng.
- Nếu sử dụng cơ sở dữ liệu phi quan hệ, việc thiết kế cơ sở dữ liệu (database design) ở bước mô hình hóa dữ liệu logic là không cần thiết. (phi quan hệ vẫn phải thiết kế)

Lecture 6. Class Design

1. Mối quan hệ Association trong thiết kế hướng đối tượng là gì?

- Mối quan hệ mà một lớp kế thừa từ lớp khác (kế thừa)
- Mối quan hệ mà một lớp chứa một thể hiện (instance) của lớp khác (parameter -> khi truyền vào thì truyền vào thể hiện của lớp đó) (dependency)
- Mối quan hệ kết nối hai lớp nhưng không ngụ ý quyền sở hữu
- Mối quan hệ mô tả luồng dữ liệu giữa các lớp

2. Operation khác với phương thức (method) như thế nào? (method là implementation của operation)

- Operation là khái niệm chung về hành vi, trong khi phương thức là hiện thực cụ thể của hành vi đó trong một lớp
- Operation là phương thức riêng tư (private), còn phương thức là hàm công khai (public)
- Operation là một hàm, trong khi phương thức định nghĩa các thuộc tính
- Operation liên quan đến dữ liệu, còn phương thức liên quan đến chức năng

3. Điều nào sau đây có thể làm cho đối tượng thay đổi trạng thái?

- Có phương thức được gọi
- Thuộc tính được cập nhật
- Mối quan hệ giữa hai đối tượng thay đổi
- Tất cả các đáp án trên

4. Mối quan hệ kế thừa (Inheritance) giúp loại bỏ sự trùng lặp như thế nào?

- Bằng cách sử dụng đối tượng từ lớp khác
- Bằng cách kế thừa mã nguồn từ lớp cha

- Bằng cách tái sử dụng phương thức của lớp khác mà không cần kế thừa
- Bằng cách giữ mã nguồn trong các phương thức riêng biệt

5. Điểm khác biệt chính giữa Association và Dependency là gì?

- Association là **quan hệ tạm thời (sai)**, còn Dependency là quan hệ lâu dài
- Dependency **không yêu cầu mũi tên (sai)**, còn Association thì luôn có
- Association **không yêu cầu (sai)** khởi tạo đối tượng ngay từ đầu, còn Dependency thì có
- Association yêu cầu có sự tham chiếu giữa các đối tượng, còn Dependency không cần (**field reference**)

6. Aggregation khác gì so với Composition?

- Aggregation yêu cầu đối tượng toàn thể phải tồn tại khi đối tượng con tồn tại
- Trong Aggregation, khi đối tượng con bị phá hủy thì đối tượng toàn thể vẫn tồn tại
- Aggregation và Composition đều không yêu cầu mũi tên chỉ ra quan hệ
- Aggregation yêu cầu đối tượng toàn thể và con phải bị phá hủy cùng nhau

7. Trong quan hệ Dependency, một lớp có thể phụ thuộc vào lớp khác theo cách nào?

- Thông qua biến cục bộ trong phương thức (**local variable**) (nếu chỉ chọn 1 -> chọn đáp này)
- Thông qua một thuộc tính (field) (**association**)
- Thông qua kế thừa từ lớp khác (**kế thừa**)
- Thông qua sử dụng phương thức tĩnh của lớp khác (**dependency**)

8. Quan hệ nào sau đây có tính lỏng lẻo nhất? (**depend < asso < aggre < compo < inheritance**)

- **Dependency**
- Association
- Aggregation
- Composition

Lecture 6 (Part 2). System Interface Design

1. Trong UML, đâu là sự khác biệt giữa một hệ thống con (subsystem) và một gói (package)?

- Subsystem không thể được tái sử dụng, trong khi Package được thiết kế để tái sử dụng.
- Subsystem chỉ có thể chứa các lớp, trong khi Package có thể chứa Subsystem khác và Package khác.
- Package được dùng để tổ chức/gom nhóm các phần tử, trong khi Subsystem không chỉ tổ chức mà còn cung cấp các hành vi thông qua giao diện (interface).
- Các phần tử bên trong Package luôn có phạm vi truy cập là **public (sai)**, còn trong Subsystem thì chỉ được có phạm vi truy cập là private.

2. Bạn đang thiết kế giao diện IPayment.

Trong các hành vi (operation) sau, hãy chọn các hành vi phù hợp để đảm bảo tính tổng quát của giao diện hệ thống: (**không phụ thuộc phương thức thanh toán**)

1. **payVNPay(Order order, VNPayConfig config); (specific quá)**
2. **pay(Order order, PaymentContext ctx);**

3. `pay(Order order);`
4. `pay(int totalAmount, String orderID);`
5. `payPayPal(Order order, PayPalConfig config);` (specific quá)

- ☐ 1
- ☒ 2
- ☒ 3
- ☒ 4
- ☐ 5

3. Giả sử bạn đang cân nhắc 2 phương án:

- **Phương án 1:** Cho `PlaceOrderController` gọi trực tiếp `VNPaySubsystem`.
- **Phương án 2:** Cho `PlaceOrderController` làm việc thông qua `IPayment`, trong khi `VNPaySubsystem` chỉ là một triển khai cụ thể của `IPayment`.

Hãy chọn các phát biểu đúng.

- ☒ Phương án 2 làm giảm độ phụ thuộc giữa nghiệp vụ đặt hàng và cổng thanh toán.
- ☐ Phương án 1 giúp hiệu năng tốt hơn vì không cần abstraction, ngoài ra thiết kế cũng dễ hiểu hơn vì gần với `VNPaySubsystem`.
- ☒ Phương án 2 giúp dễ thay thế subsystem mới sau này có một cổng thanh toán mới được tích hợp vào hệ thống.
- ☐ Phương án 1 phù hợp nhất, bởi hệ thống luôn chỉ dùng một cổng thanh toán cố định và việc mở rộng là rất khó xảy ra.

4. Bạn đang phát triển hệ thống “SmartFarm” gồm các thiết bị: cảm biến nhiệt độ, độ ẩm, ánh sáng. Ban đầu lớp biên **SensorBoundary** vừa đọc dữ liệu, vừa xử lý, vừa gửi về server.

Khi nào nên tách lớp biên này thành subsystem riêng, ví dụ **SensorSubsystem**?

- ☒ Khi số lượng loại cảm biến tăng lên.
- ☒ Khi giao tiếp giữa các cảm biến và hệ thống trung tâm phức tạp dần.
- ☐ Khi chỉ có một cảm biến duy nhất, hoạt động ổn định.
- ☐ Khi hệ thống không có kế hoạch mở rộng thêm thiết bị.
- ☒ Khi bạn muốn thay đổi giao thức kết nối mà không chạm đến phần xử lý nghiệp vụ.

5. Những đặc điểm nào sau đây là quan trọng khi thiết kế một hệ thống con (subsystem) có khả năng tái sử dụng?

- ☒ Tính đóng gói cao.
- ☐ Tính phụ thuộc vào hệ thống khác.
- ☒ Giao diện (Interface) rõ ràng và ổn định.
- ☒ Sự linh hoạt trong thiết kế nội bộ.

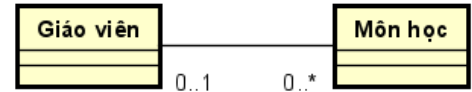
1. Biểu đồ thực thể liên kết (Entity Relationship Diagram) là đầu ra của bước nào trong quá trình mô hình hóa dữ liệu?

☒ Mô hình hóa dữ liệu mức khái niệm (Conceptual Data Modelling).

☐ Mô hình hóa dữ liệu mức vật lý (Physical Data Modelling). (triển khai DB trên DBMS)

☐ Mô hình hóa dữ liệu mức logic (Logical Data Modelling). (DB Design) (bắt đầu phụ thuộc DBMS)

2. Từ biểu đồ lớp sau, hãy chọn các phát biểu đúng về mối quan hệ giữa "Giáo viên" và "Môn học":



☐ Một giáo viên chỉ được dạy tối đa một môn học. (do dấu * bên phải -> sai)

☒ Một môn học được dạy bởi nhiều nhất một giáo viên. (do số 1 bên trái)

☒ Một giáo viên có thể không dạy môn học nào. (do số 0 bên phải)

☐ Một môn học có thể được dạy bởi nhiều hơn một giáo viên. (do số 1 bên trái -> sai)

3. Một máy bán hàng tự động có thể ở các trạng thái **Idle** (chờ), **HasMoney** (đã nhận tiền), **Dispensing** (đang xuất hàng), và **OutOfStock** (hết hàng). Khi người dùng đưa tiền vào, máy chuyển từ Idle sang HasMoney. Khi chọn sản phẩm, máy chuyển sang Dispensing, sau đó quay lại Idle. Nếu hết hàng, máy chuyển sang OutOfStock. Trong biểu đồ máy trạng thái (state machine diagram), yếu tố nào mô tả sự kiện khiến máy bán hàng thay đổi trạng thái?

☐ State (Trạng thái)

☒ Transition (Chuyển tiếp)

☐ Initial State (Trạng thái đầu)

☐ Final State (Trạng thái cuối)

4. Hãy chọn phát biểu đúng.

☐ Mọi lớp phân tích được phát hiện trong giai đoạn phân tích use case đều được giữ lại trong giai đoạn thiết kế chi tiết.

☒ Nếu sự thay đổi hành vi hoặc cấu trúc của một lớp dẫn đến sự thay đổi của lớp còn lại thì hai lớp đó nên được đặt trong cùng một gói (package).

☒ Khi một lớp biên quá phức tạp, nên tái cấu trúc nó thành một hệ thống con (subsystem) trong giai đoạn thiết kế chi tiết.

☐ Giao diện người dùng (user interface) chỉ bao gồm giao diện đồ họa (graphical user interface). (quay lại lecture 4)

5. Ở những đoạn mã nguồn nào sau đây, mối quan hệ giữa lớp A và lớp B là dependency?

Dependency có 3 loại: (A phụ thuộc vào > B)

- Local Variable (biến ở trong 1 hàm của class A)

- Parameter (tham số truyền vào của 1 hàm trong A)

- Public static (gọi 1 phương thức (toàn cục) của B trong 1 hàm của A) static = gọi hàm mà ko phải khởi tạo biến của B

Aggregation:

- Không được khởi tạo trong Constructor

Composition:

- Khởi tạo ngoài method
- Khởi tạo với constructor

<pre> class A { public void print() { System.out.println("This is class A."); } } class B { A a; public B(A a) { this.a = a; } public void method1() { a.print(); } } </pre>	<pre> class A { public void print() { System.out.println("This is class A."); } } class B { public void method1(A a) { a.print(); } } </pre>
☐ Tùy chọn 1 (attribute -> asso)	☐ Tùy chọn 2 (parameter)
<pre> class A { public void print() { System.out.println("This is class A."); } } class B { public void method1() { A a = new A(); a.print(); } } </pre>	<pre> class A { public void print() { System.out.println("This is class A."); } } class B { final A a; public B(A a) { this.a = a; } public void method1() { A a = new A(); a.print(); } } </pre>
☐ Tùy chọn 3 (local variable)	☐ Tùy chọn 4 (aggregation) (nếu this.a = new a thì là compo, không được truyền A a = ở parameter)

6. Cho tình huống sau:

- Lớp **PlaceOrderController** chứa phương thức **calculateShippingFee()**, thuộc package **controllers**.
- Lớp **Order** chứa phương thức **setDelivery()**, thuộc package **models**.
- Lớp **DeliveryScreen** chứa phương thức **validateDeliveryInfo()**, thuộc package **views**.
- Lớp **RushDeliveryScreen** thuộc package **views**.

- Lớp **VietQRRequest** chứa phương thức **buildRequestString()**, thuộc package **subsystem.vietqr**.

Tình huống nào sau đây là hợp lệ (giả sử các lớp trên đều có chỉ định truy cập là public)?

- ☐ Phương thức **setDelivery()** có chỉ định truy cập là **public**. Trong lớp **PlaceOrderController**, gọi **order.setDelivery()** (với **order** là đối tượng của lớp **Order**).
- ☐ Phương thức **calculateShippingFee()** có chỉ định truy cập là **private**. Trong lớp **DeliveryScreen**, gọi **controller.calculateShippingFee()** (với **controller** là đối tượng của lớp **PlaceOrderController**).
- ☐ Phương thức **buildRequestString()** có chỉ định truy cập là default. Trong lớp **PlaceOrderController**, gọi **request.buildRequestString()** (với **request** là đối tượng của lớp **VietQRRequest**). (**!= package**)
- ☐ Phương thức **validateDeliveryInfo()** có chỉ định truy cập là **protected**. Trong lớp **RushDeliveryScreen**, gọi **deliveryScreen.validateDeliveryInfo()** (với **deliveryScreen** là đối tượng của lớp **DeliveryScreen**). (**cùng package**)

Default: cùng package

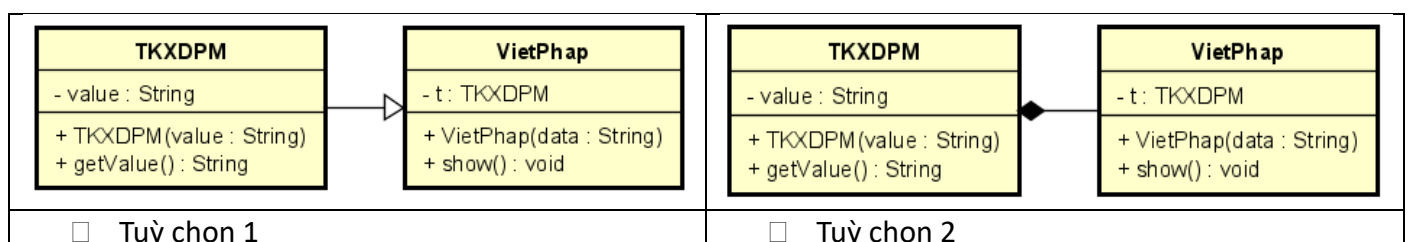
Protected: cùng package & cùng cây kế thừa

7. Hãy chọn biểu đồ lớp phù hợp nhất tương ứng với đoạn mã nguồn sau:

```
public class TKXDPM {
    private String value;
    public TKXDPM(String value) {
        this.value = value;
    }
    public String getValue() {
        return value;
    }
}

public class VietPhap {
    private final TKXDPM t;
    public VietPhap(String data) {
        this.t = new TKXDPM(data);
    }

    public void show() {
        System.out.println(t.getValue());
    }
}
```



TKXDPM - value : String + TKXDPM(value : String) + getValue() : String VietPhap - t : TKXDPM + VietPhap(data : String) + show() : void	TKXDPM - value : String + TKXDPM(value : String) + getValue() : String VietPhap - t : TKXDPM + VietPhap(data : String) + show() : void
☐ Tuỳ chọn 3 (composition) (VP mất -> TKXDPM mất)	☐ Tuỳ chọn 4

8. Cho đoạn mô tả: Khi người dùng nhấn vào bất kỳ sản phẩm nào trên màn hình "Trang chủ", hệ thống sẽ hiển thị màn hình "Thông tin chi tiết sản phẩm" dưới dạng popup. Trong thời gian popup mở, màn hình "Trang chủ" sẽ bị vô hiệu hóa và không thể thao tác cho đến khi popup được đóng lại.

Biểu đồ dịch chuyển màn hình nào sau đây là phù hợp nhất với mô tả trên?

Trang chủ Thông tin chi tiết sản phẩm	Trang chủ Thông tin chi tiết sản phẩm
☐ Tuỳ chọn 1 (Khi trang chủ được khởi tạo thì popup chưa được khởi tạo, khi người dùng bấm vào sản phẩm thì popup mới được khởi tạo, khi popup mất đi thì trang chủ không mất đi) (aggragation)	☐ Tuỳ chọn 2
Trang chủ Thông tin chi tiết sản phẩm	Trang chủ Thông tin chi tiết sản phẩm
☐ Tuỳ chọn 3	☐ Tuỳ chọn 4

9. Trong UML, ký hiệu nào dưới đây thể hiện chỉ định truy cập protected?

- ☐ - private
- ☐ + public
- ☒ #
- ☐ ~ default

10. Đây là sự khác biệt chính giữa một hệ thống con (subsystem) và một gói (package)?

- ☐ Hệ thống con không thể được tái sử dụng, trong khi gói được thiết kế để tái sử dụng.
- ☐ Hệ thống con chỉ có thể chứa các lớp, trong khi gói có thể chứa cả các hệ thống con và các gói khác.
- ☒ **Gói được dùng để tổ chức các phần tử, trong khi hệ thống con không chỉ tổ chức mà còn cung cấp hành vi thông qua các giao diện (interface).**
- ☐ Các phần tử bên trong một gói luôn có phạm vi truy cập public, trong khi các phần tử trong một hệ thống con chỉ có phạm vi truy cập private.

Quiz 27/11: Lập trình và Kiểm thử

1. Phát biểu nào sau đây là đúng khi nói về kiểm thử hộp đen (Black-box testing) và kiểm thử hộp trắng (White-box testing)?

- ☒ **Kiểm thử hộp đen và Kiểm thử hộp trắng thực chất là các kỹ thuật để thiết kế test case.**
- ☐ Kiểm thử hộp đen yêu cầu lập trình viên cần có hiểu biết về toàn bộ logic bên trong của mã nguồn để thiết kế test case. (hộp trắng)

- ☐ Các phương pháp kiểm thử hộp đen tiêu biểu là bao phủ lệnh và bao phủ nhánh. (hộp trắng)
- ☒ Kiểm thử hộp đen chỉ quan tâm đến đầu vào và đầu ra mong muốn của đơn vị cần kiểm thử, không quan tâm đến chi tiết logic bên trong.
- ☒ Nhờ sử dụng phương pháp phân vùng tương đương (trong kiểm thử hộp đen) mà các test case được thiết kế đủ để bao quát tất cả các trường hợp có thể xảy ra.

2. Trong các quy trình sau, quy trình nào được thực hiện bởi kiểm thử viên (tester)?

- ☐ Unit Test (Developer)
- ☒ Integration Test
- ☒ System Test
- ☐ User Acceptance Test (User)

3. Chương trình của bạn cần kiểm thử sự tương tác giữa logic của phần và một hệ thống bên thứ ba (ví dụ: VNPAY). Loại kiểm thử nào dưới đây là phù hợp nhất?

- ☐ Unit Test
- ☒ Integration Test
- ☐ System Test
- ☐ User Acceptance Test

4. Điểm khác biệt giữa kiểm thử đơn vị (Unit Test) và kiểm thử tích hợp (Integration Test) là gì?

- ☒ Kiểm thử đơn vị tập trung vào kiểm thử từng thành phần độc lập, trong khi kiểm thử tích hợp kiểm tra sự tương tác giữa các thành phần.
- ☐ Kiểm thử tích hợp kiểm tra từng đơn vị (unit) riêng lẻ, trong khi kiểm thử đơn vị kiểm tra cả hệ thống tích hợp.
- ☒ Kiểm thử đơn vị thường được thực hiện bởi lập trình viên, còn kiểm thử tích hợp thường được thực hiện bởi kiểm thử viên.
- ☐ Cả kiểm thử đơn vị và kiểm thử tích hợp đều không cần thiết nếu đã có kiểm thử cho toàn bộ hệ thống.

5. Cho đoạn mã như sau. Hãy chọn phát biểu đúng?

```
if user.age > 18 {
    System.out.println("Adult");
}
```

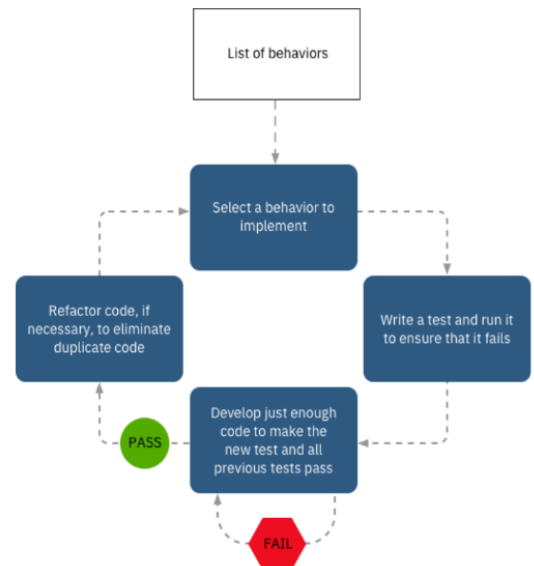
- ☐ Giữ nguyên code như hiện tại vì đã đủ rõ ràng.
- ☒ Nên đặt 18 thành hằng số có ý nghĩa, ví dụ ADULT_AGE = 18.
- ☐ Có thể thêm comment giải thích “18 là tuổi trưởng thành”.
- ☐ Nên viết code gọn trong 1 dòng cho ngắn gọn.

6. Việc viết mã nguồn sạch (clean code) là vô cùng quan trọng trong xây dựng phần mềm, nhằm đảm bảo code dễ đọc, dễ hiểu, dễ mở rộng và dễ bảo trì. Những việc nào sau đây nên được áp dụng để có một mã nguồn sạch?

- ☐ Viết càng nhiều comment càng tốt để mã nguồn trở nên dễ hiểu.

- ❑ Khi đặt tên biến, cần đặt tên sao cho tên đó mô tả rõ ràng ý nghĩa hoặc nhiệm vụ của biến đó, tránh những tên quá chung chung và khó hiểu.
- ❑ Nên tuân theo các quy ước khi đặt tên biến (ví dụ: biến i nên được sử dụng cho chỉ số trong các vòng lặp).
- ❑ Các hàm nên tập trung vào một nhiệm vụ duy nhất.
- ❑ Tránh trùng lặp mã nguồn: Nếu một đoạn mã có sự trùng lặp (xuất hiện từ 2 lần trở lên), cần nghĩ đến cách tối ưu nó như tách thành hàm hoặc module riêng. (Design pattern)

7. Trong quy trình Test-Driven Development (TDD), các bước nào sau đây là đúng?



- ❑ Lựa chọn một module trong chương trình để thực hiện kiểm thử đơn vị.
- ❑ Tiến hành thiết kế các bộ test case trước khi viết mã nguồn chi tiết.
- ❑ Viết mã nguồn đầy đủ ngay từ đầu để đảm bảo tất cả bộ test case đều thành công.
- ❑ Triển khai phần khung mã nguồn tối thiểu để tất cả các bộ test case đã thiết kế có thể chạy thành công.
- ❑ Tiếp tục chỉnh sửa mã nguồn cho đến khi tất cả các bộ test case chạy thành công, đồng thời làm cho mã nguồn trở nên tốt hơn so với ban đầu.
- ❑ Tiến hành viết mã nguồn trước, sau đó mới xây dựng các bộ test case dựa trên logic thực tế của đoạn mã đã viết. (sai do đề là test-driven)

8. Sắp xếp các quy trình con sau đây theo đúng thứ tự trong quy trình phát triển phần mềm đã được giới thiệu trong học phần này? (Đánh số)

Quy trình con	Thứ tự
Software Requirement Analysis (từ yêu cầu -> SRS, UC diag)	1
Software Architectural Design (từ UC -> seq,...)	2
Software Detailed Design (class diagram)	3
Software Construction (code)	4

Quiz 11/12: Coupling & Cohesion

1. Hãy sắp xếp các mức độ coupling (mức độ phụ thuộc giữa các module) theo thứ tự từ chặt nhất (tight coupling) đến lỏng nhất (loose coupling) (lỏng là tốt)

Mức độ coupling	Thứ tự
Content coupling	1
Common coupling	2

Control coupling	3
Stamp coupling	4
Data coupling	5

1. **Content Coupling (Tệ nhất):** Module A truy cập trực tiếp hoặc sửa đổi dữ liệu nội bộ/code của Module B (vi phạm Encapsulation).
2. **Common Coupling:** Các module chia sẻ dữ liệu toàn cục (Global data).
3. **Control Coupling:** A truyền cờ (flag) để điều khiển luồng xử lý của B (A phải biết logic bên trong B).
Vd: if payment 2 = vnpay do else if ...
4. **Stamp Coupling:** Truyền cả một cấu trúc dữ liệu lớn (Object/Struct) nhưng chỉ dùng vài trường trong đó. Vd: xử lý bằng DTO
5. **Data Coupling (Tốt nhất):** Chỉ truyền đúng dữ liệu cần thiết qua tham số hàm (primitive types hoặc data đơn giản).

2. Hãy sắp xếp mức độ cohesion (mức độ liên quan giữa các thành phần trong 1 module) theo thứ tự từ lỏng nhất (loose cohesion) đến chặt nhất (tight cohesion) (càng chặt càng tốt)

Mức độ cohesion	Thứ tự
Coincidental cohesion	1
Logical cohesion	2
Temporal cohesion	3
Procedural cohesion	4
Communicational cohesion	5
Sequential cohesion	6
Functional cohesion	7

1. **Coincidental (Tệ nhất):** Gom nhóm ngẫu nhiên, không liên quan gì nhau (Ví dụ: Class Utils chứa hàm Print() và CalculateTax()).
2. **Logical:** Gom nhóm vì cùng loại hành động (Ví dụ: Hàm InputAll() xử lý nhập chuột, phím, disk...), nhưng logic khác nhau.
3. **Temporal:** Gom nhóm vì chúng xảy ra cùng một thời điểm (Ví dụ: Initialization() khởi tạo file, database, biến...).
4. **Procedural:** Các thành phần theo một trình tự thực thi cụ thể.
5. **Communicational:** Các thành phần thao tác trên cùng một dữ liệu đầu vào/ra (Ví dụ: UpdateRecord và WriteToLog cùng dùng CustomerData).
6. **Sequential:** Output của phần này là Input của phần kia (Dây chuyền sản xuất). (Procedural + putput của procedure trước là input của procedure sau)
7. **Informational:** Một module thực hiện nhiều hành động, mỗi hành động có điểm vào riêng, code độc lập, tất cả thao tác trên cùng cấu trúc dữ liệu (Thường thấy ở ADT - Abstract Data Type).
8. **Functional (Tốt nhất):** Toàn bộ module chỉ thực hiện **một nhiệm vụ duy nhất** và rõ ràng (Ví dụ: Hàm CalculateSin(x)).

3. Đây là biểu hiện của Coincidental Cohesion?

- ☐ Các thành phần trong module không có bất kỳ liên quan gì đến nhau, chỉ được nhóm chung một chỗ vì ngẫu nhiên.
- ☐ Module thực hiện nhiều chức năng khác nhau mà không có sự liên kết logic.
- ☐ Module chứa các phương thức cùng hướng đến hoàn thành một trách nhiệm duy nhất. (functional)
- ☐ Các thành phần trong module được liên kết với nhau bởi các thuộc tính/dữ liệu chung. (communicational)

4. Đây là nhược điểm của Stamp Coupling? Stamp Coupling ~ truyền dư thừa dữ liệu (chỉ xảy ra được khi chẳng nhận biết được cấu trúc dữ liệu của thằng gửi)

- ☐ Truyền dữ liệu dư thừa không cần thiết.
- ☐ Gây khó hiểu cho client khi sử dụng do lộ những chi tiết không cần thiết.
- ☐ Tăng rủi ro xảy ra lỗi do dữ liệu không phù hợp.
- ☐ Dễ dàng tái sử dụng các module liên quan.

5. Khi nào một module được xem là đang ở mức Logical Cohesion?

- Khi các phần trong module cùng thực hiện một chuỗi xử lý theo thứ tự nhất định, đầu ra của phần này là đầu vào của phần kế tiếp. (sequential)
- Khi các phần trong module thực hiện các công việc thuộc cùng một nhóm logic, nhưng được chọn để chạy bằng tham số điều khiển.
- Khi tất cả các phần trong module cùng phục vụ một mục tiêu duy nhất, không phụ thuộc vào nhau. (functional)
- Khi các phần trong module không liên quan gì đến nhau, chỉ được nhóm lại ngẫu nhiên. (coincidental)

6. Cho đoạn mã nguồn như bên dưới. Lớp PaymentProcessor đang ở mức độ coupling nào?

```

public class PaymentProcessor {

    public void processPayment(String paymentType, double amount) {
        if (paymentType.equals("CreditCard")) {
            processCreditCard(amount);
        } else if (paymentType.equals("PayPal")) {
            processPayPal(amount);
        } else if (paymentType.equals("Momo")) {
            processMomo(amount);
        } else {
            System.out.println("Payment type not supported");
        }
    }

    private void processCreditCard(double amount) {
        System.out.println("Processing credit card payment: " + amount);
    }

    private void processPayPal(double amount) {
        System.out.println("Processing PayPal payment: " + amount);
    }

    private void processMomo(double amount) {
        System.out.println("Processing Momo payment: " + amount);
    }
}

```

- Content coupling
- Common coupling
- Control coupling
- Stamp coupling
- Data coupling

7. Giả sử bạn có lớp LibraryManager gồm các phương thức:

- addBook() – đảm nhiệm logic thêm sách mới
- removeBook() – đảm nhiệm logic xóa sách
- searchBookByTitle() – đảm nhiệm logic tìm kiếm sách bằng tên
- printMonthlyReport() – đảm nhiệm việc in báo cáo doanh thu hàng tháng

Hãy chọn phát biểu đúng nhất về mức độ cohesion của lớp LibraryManager.

- Functional cohesion – vì tất cả đều nằm trong cùng lớp.
- Functional cohesion – vì các phương thức hỗ trợ cùng một mục tiêu duy nhất.
- Low cohesion – vì có chức năng quản lý sách lẫn in báo cáo doanh thu không liên quan. (1,2 là CRUD cùng phục vụ quản lý sách, 3,4 không liên quan)

- Low cohesion – vì mỗi một phương thức làm một việc khác nhau.

Quiz 18/12: SOLID Design Principles

1. Làm thế nào để áp dụng Nguyên lý Đảo ngược Phụ thuộc (Dependency Inversion Principle) trong việc thiết kế chương trình nhắc nhở nghỉ giải lao?

- Để lớp Timer trực tiếp phụ thuộc vào các lớp cụ thể như TimeToStretch hoặc TimeToRelax.
- Để lớp TimeToStretch kế thừa từ lớp Timer và xử lý logic của Timer.
- Sử dụng một interface hoặc lớp trừu tượng chung như TimeToAction để Timer tương tác với các hành động nhắc nhở thông qua abstraction.
- Viết tất cả logic nhắc nhở nghỉ giải lao trong lớp Timer mà không sử dụng các lớp khác.

2. Đoạn mã nguồn sau vi phạm những nguyên lý SOLID nào?

```
public class OrderService {  
  
    public void processOrder(Order order, String paymentType) {  
        if (paymentType.equals("CREDIT_CARD")) {  
            // xử lý thanh toán thẻ  
            System.out.println("Pay by credit card");  
        } else if (paymentType.equals("PAYPAL")) {  
            // xử lý thanh toán PayPal  
            System.out.println("Pay by PayPal");  
        }  
  
        // lưu đơn hàng vào DB  
        System.out.println("Save order to database");  
  
        // gửi email xác nhận  
        System.out.println("Send confirmation email");  
    }  
}
```

- ☐ Đoạn mã vi phạm Single Responsibility Principle (SRP) vì OrderService vừa xử lý thanh toán, vừa lưu dữ liệu, vừa gửi email.
- ☐ Đoạn mã vi phạm Open/Closed Principle (OCP) vì mỗi khi thêm phương thức thanh toán mới, ta phải sửa lại phương thức processOrder.
- ☐ Đoạn mã vi phạm Liskov Substitution Principle (LSP) vì paymentType được truyền dưới dạng String.
- ☐ Đoạn mã vi phạm Dependency Inversion Principle (DIP) vì OrderService phụ thuộc trực tiếp vào chi tiết xử lý thanh toán cụ thể.
- ☐ Đoạn mã vi phạm Interface Segregation Principle (ISP) vì OrderService có quá nhiều phương thức public.

3. Những khẳng định nào sau đây KHÔNG đúng theo nguyên lý Dependency Inversion Principle (DIP)?

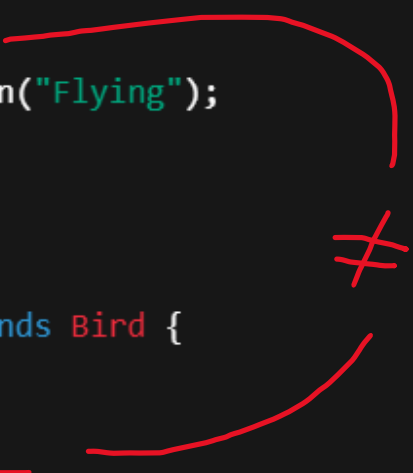
- ☐ Các module cấp cao phải phụ thuộc trực tiếp vào các module cấp thấp.
- ☐ Các module cấp cao và thấp nên phụ thuộc vào các abstraction thay vì phụ thuộc vào nhau. (đúng)
- ☐ Các module cấp cao không nên phụ thuộc vào các module cấp thấp, mà nên phụ thuộc vào một abstraction. (đúng)
- ☐ Abstraction nên phụ thuộc vào chi tiết.

Cấp cao ~ interface, lớp cha

Cấp thấp ~ extend từ lớp cha

4. Cho đoạn mã nguồn sau. Hãy chọn các khẳng định đúng.

```
public class Bird {  
    public void fly() {  
        System.out.println("Flying");  
    }  
}  
  
public class Penguin extends Bird {  
    @Override  
    public void fly() {  
        throw new UnsupportedOperationException("Penguins can't fly");  
    }  
}
```



- ☐ Đoạn mã vi phạm Liskov Substitution Principle (LSP) vì Penguin không thể thay thế hoàn toàn cho Bird.
- ☐ Đoạn mã vi phạm Single Responsibility Principle (SRP) vì lớp Bird làm quá nhiều việc.
- ☐ Có thể cải thiện bằng cách tách khả năng bay thành một interface riêng, ví dụ Flyable.
- ☐ Đoạn mã vi phạm Open/Closed Principle (OCP) vì phương thức fly() bị override.
- ☐ Có thể cải thiện bằng cách loại bỏ kế thừa và sử dụng composition thay cho inheritance.

5. Hãy chọn các phát biểu đúng.

- ☐ Một hệ thống tuân thủ tốt nguyên lý SOLID sẽ giúp mã nguồn dễ bảo trì và mở rộng hơn khi yêu cầu thay đổi.
- ☐ Chỉ cần sử dụng interface trong chương trình thì chắc chắn không vi phạm nguyên lý Dependency Inversion.
- ☐ Open/Closed Principle khuyến khích việc sửa đổi trực tiếp mã nguồn hiện có khi thêm chức năng mới.
- ☐ Interface Segregation Principle cho rằng nhiều interface nhỏ, chuyên biệt thường tốt hơn một interface lớn, tổng quát.

6. Nguyên lý Interface Segregation Principle (ISP) khuyến khích những việc làm nào sau đây?

- ☒ Sử dụng các interface nhỏ và chuyên biệt thay vì một interface lớn.
- ☒ Tách các interface chung thành các interface riêng biệt cho từng nhóm client.
- ☐ Sử dụng các interface phức tạp để giảm số lượng lớp.
- ☐ Tạo một interface chung cho tất cả các client.

7. Cho đoạn mã nguồn sau, những khẳng định nào đúng?

```
public class FileLogger {  
    public void log(String message) {  
        System.out.println("Write log to file: " + message);  
    }  
}  
  
public class OrderController {  
  
    private FileLogger logger = new FileLogger();  
  
    public void createOrder() {  
        // xử lý tạo đơn hàng  
        logger.log("Order created");  
    }  
}
```

- ☒ OrderController đang phụ thuộc trực tiếp vào một chi tiết cài đặt cụ thể, khiến việc thay đổi cách ghi log trở nên khó khăn.
- ☒ Đoạn mã vi phạm Dependency Inversion Principle (DIP) vì module cấp cao phụ thuộc vào module cấp thấp.
- ☒ Có thể cải thiện bằng cách để OrderController phụ thuộc vào một abstraction chung cho việc ghi log.
- ☐ Đoạn mã vi phạm Interface Segregation Principle (ISP) vì FileLogger chỉ có một phương thức.
- ☒ Thiết kế hiện tại làm giảm khả năng kiểm thử (testability) của OrderController. (Muốn thử file logger thì phải thử cả OrderController)

8. Single Responsibility Principle (SRP) yêu cầu rằng một lớp phải đảm bảo những tiêu chí nào?

- ☒ Chỉ nên chịu trách nhiệm về một chức năng duy nhất.
- ☐ Nên chịu trách nhiệm về nhiều chức năng để linh hoạt hơn.
- ☐ Không được thay đổi trong bất kỳ hoàn cảnh nào.
- ☒ Chỉ nên tồn tại một lý do duy nhất để nó phải thay đổi.

9. Nguyên lý Open/Closed Principle (OCP) nói rằng các lớp cần phải đảm bảo những tiêu chí nào?

- ☐ Phải luôn luôn mở để thay đổi (open for modification).
- ☒ Phải luôn luôn đóng đối với thay đổi, mở cho sự mở rộng (close for modification, open for extension).

- ☐ Không bao giờ được mở rộng vì gây ra nhiều rắc rối cho hệ thống.
- ☐ Phải có khả năng mở rộng sao cho thiết kế và mã nguồn hiện tại không có sự sửa đổi.

10. Theo nguyên lý thay thế Liskov (Liskov Substitution Principle LSP), những khẳng định nào sau đây là đúng?

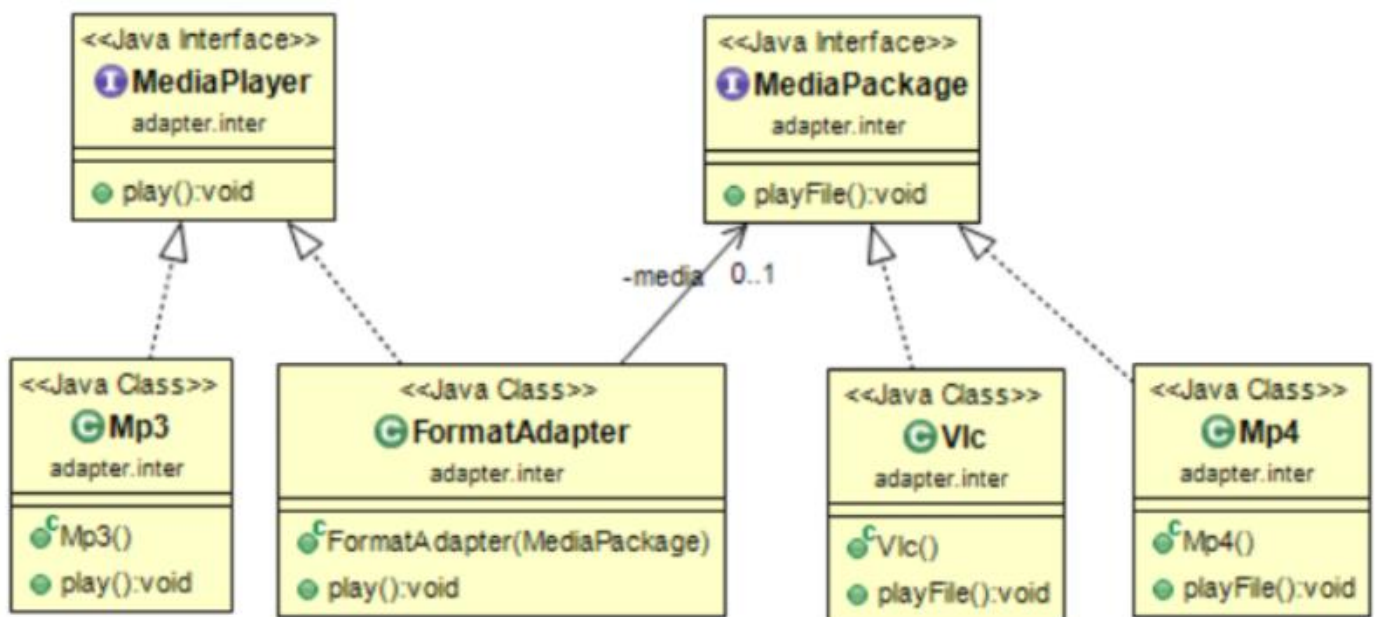
- ☐ Lớp con phải có thể thay thế lớp cha mà không gây ra lỗi.
- ☐ Lớp con có thể thay thế lớp cha nhưng phải thay đổi hành vi của lớp cha.
- ☐ Lớp con có thể thay thế lớp cha mà không làm thay đổi tính đúng đắn của chương trình.
- ☐ Lớp con không nên thay thế lớp cha.

Quiz 25/12: Design Pattern (Singleton, Adapter, Template Method)

1. Đây là một ứng dụng thực tế của Singleton Pattern?

- ☐ Cho phép hai thành phần có giao diện khác nhau vẫn có thể làm việc được với nhau.
- ☐ Xây dựng một bộ khung xử lý chung, trong đó một số bước được để cho các lớp con tự cài đặt.
- ☐ Quản lý việc kết nối cơ sở dữ liệu sao cho trong toàn bộ chương trình chỉ tồn tại một đối tượng kết nối duy nhất.
- ☐ Tạo ra các đối tượng khác nhau dựa trên tham số đầu vào mà không cần biết trước lớp cụ thể của đối tượng.

2. Chọn các phát biểu đúng về thiết kế sau đây (theo những gì đã được học trên lớp).



- ☐ Lớp FormatAdapter phải có một thuộc tính có kiểu là MediaPackage. Giả sử thuộc tính đó tên là mp.
- ☐ Trong phương thức play(), phải gọi mp.playFile() để chạy các thiết bị như Vlc hay Mp4.
- ☐ Đây là cách triển khai của Adapter Pattern theo hướng tiếp cận lớp (class).
- ☐ Đây là cách triển khai của Adapter Pattern theo hướng tiếp cận đối tượng (object). (do sử dụng composition thay vì inheritance)
- ☐ Lớp FormatAdapter không nhất thiết phải chứa phương thức play(). Có thể đặt một tên khác cho phương thức này.
- ☐ Không cần thiết phải tạo lớp FormatAdapter, do có thể sửa trực tiếp MediaPlayer, MediaPackage.

3. Cho đoạn mã nguồn sau. Hãy chọn các phát biểu đúng.

```
abstract class DataProcessor {
    // template method
    public final void process() {
        readData();
        processData();
        writeData();
    }

    protected abstract void readData();

    protected abstract void processData();

    protected void writeData() {
        System.out.println("Write data to file");
    }
}

class CsvDataProcessor extends DataProcessor {
    @Override
    protected void readData() {
        System.out.println("Read CSV data");
    }

    @Override
    protected void processData() {
        System.out.println("Process CSV data");
    }
}
```

- ☒ Phương thức process() xác định sẵn trình tự xử lý và không cho phép lớp con thay đổi thứ tự các bước.
- ☐ Lớp CsvDataProcessor có thể ghi đè phương thức process() để thay đổi luồng xử lý.
- ☒ Việc đánh dấu process() là final giúp đảm bảo cấu trúc thuật toán không bị phá vỡ (đồng nghĩa với việc phương thức process() không thể bị ghi đè).
- ☒ Các lớp con như CsvDataProcessor chỉ cần cài đặt những bước thay đổi, còn các bước dùng chung có thể được tái sử dụng.
- ☐ Mẫu thiết kế này cho phép lựa chọn thuật toán xử lý tại runtime thông qua việc truyền đối tượng vào constructor. (strategy)

4. Tình huống nào sau đây gợi nhớ đến việc sử dụng Adapter Pattern?

- ☐ Khi muốn đảm bảo chỉ tồn tại một đối tượng dùng chung để quản lý tài nguyên toàn cục.
- ☐ Khi cần mở rộng hoặc thay đổi thuật toán xử lý mà không làm ảnh hưởng đến cấu trúc tổng thể của chương trình.
- ☒ Khi có các thành phần đã tồn tại với giao diện khác nhau, nhưng vẫn cần tích hợp chúng vào hệ thống mà không chỉnh sửa mã nguồn hiện có.

- Khi muốn kiểm soát quá trình tạo đối tượng và trì hoãn việc khởi tạo cho đến khi thật sự cần.

5. Vì sao trong một số tình huống đặc thù, người ta cần áp dụng Singleton Pattern?

- Để tạo ra nhiều đối tượng giống nhau nhằm tăng khả năng tái sử dụng mã nguồn.
- Để đảm bảo chỉ tồn tại một thể hiện duy nhất của một lớp, giúp kiểm soát truy cập đến tài nguyên dùng chung và tránh các xung đột không cần thiết.
- Để cho phép các lớp con thay đổi hành vi của phương thức mà không ảnh hưởng đến cấu trúc chung.
- Để tách quá trình tạo đối tượng khỏi phần sử dụng đối tượng, giúp mã nguồn linh hoạt hơn.

6. Theo sự phân chia của GoF, Singleton Pattern thuộc nhóm nào sau đây?

- Creation Pattern
- Structure Pattern
- Behaviour Pattern

7. Hai đoạn mã nguồn sau minh họa các cách triển khai của Singleton Pattern. Trong trường hợp nào cách thứ nhất (đoạn mã nguồn phía trên) sẽ được ưu tiên sử dụng?

Cách 1: Khởi tạo sẵn

Cách 2: Khởi tạo khi yêu cầu

```
public class Singleton {
    private static final Singleton instance
        = new Singleton(); // Private constructor prevents
                          // instantiation from other classes
    private Singleton() { }
    public static Singleton getInstance() {
        return instance;
    }
}
```

```
public class Singleton {
    private static Singleton _instance;
    private Singleton() { }
    public static synchronized Singleton getInstance() {
        if (null == _instance) {
            _instance = new Singleton();
        }
        return _instance;
    }
}
```

- Khi đối tượng có thể không bao giờ được sử dụng và cần tiết kiệm tối đa tài nguyên hệ thống.
- Khi việc khởi tạo đối tượng tốn thời gian, nhưng đối tượng đó chắc chắn sẽ được sử dụng, nhằm tránh độ trễ cho client đầu tiên.
- Khi cần trì hoãn việc tạo đối tượng để giảm thời gian khởi động của chương trình.
- Khi muốn cho phép nhiều luồng tạo đối tượng song song để tăng hiệu năng cho chương trình.

8. Trong ví dụ về BeverageMachine, phương thức nào nên được định nghĩa là abstract?

- boilWater()

- ☒ addTopping()
- ☐ getCup()
- ☐ pourInCup()

9. Theo sự phân chia của GoF, Adapter Pattern thuộc nhóm nào sau đây?

- ☐ Creation Pattern (singleton, factory)
- ☒ Structure Pattern
- ☐ Behaviour Pattern (strategy, template)

10. Gang of Four (GoF) đã phân loại các Design Patterns thành mấy nhóm chính?

- ☐ 1
- ☐ 2
- ☒ 3
- ☐ 4

Quiz cuối cùng: Design Pattern (Factory, Strategy, Observer)

1. Cho đoạn mã nguồn sau. Hãy chọn các phát biểu đúng.

```
public class PizzaStore {
    Pizza orderPizza(String type) {
        Pizza pizza;
        if (type.equals("cheese")) {
            pizza = new CheesePizza();
        } else if (type.equals("greek")) {
            pizza = new GreekPizza();
        } else if (type.equals("pepperoni")) {
            pizza = new PepperoniPizza();
        }
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
    }
}
```

- ☒ Thiết kế hiện tại đang ở mức control coupling, do biến type như một cờ điều khiển quyết định loại pizza được tạo ra.

- ❑ Thiết kế hiện tại không vi phạm nguyên lý O (Open Closed Principle) nếu tương lai có thêm loại pizza mới.
- ❑ Thiết kế hiện tại vi phạm nguyên lý D (Dependency Inversion), do lớp PizzaStore phụ thuộc trực tiếp vào các lớp con tương ứng với các loại pizza.
- ❑ Để cải thiện thiết kế này, có thể sử dụng mẫu thiết kế template method. (factory)
- ❑ Thiết kế hiện tại đang ở mức coincidental cohesion do các hành động như prepare, bake... được thực hiện một cách tuần tự, không liên quan gì đến nhau.

2. Tình huống nào dưới đây gợi nhớ đến cách sử dụng của Factory Method Pattern?

- Một lớp định nghĩa sẵn **khung xử lý tổng quát** của một thuật toán, nhưng cho phép các lớp con ghi đè một số bước cụ thể mà không thay đổi cấu trúc chung của thuật toán. (template)
- Một lớp chỉ cho phép **tồn tại duy nhất một đối tượng trong suốt vòng đời** chương trình và cung cấp một điểm truy cập toàn cục đến đối tượng đó. (singleton)
- Một lớp khai báo một phương thức tạo đối tượng, nhưng việc quyết định khởi tạo lớp concrete nào được giao cho các lớp con, tùy theo ngữ cảnh sử dụng. (factory)
- Một lớp đóng vai trò trung gian để chuyển đổi giao diện của một lớp có sẵn thành giao diện mà client mong muốn, giúp các lớp không tương thích có thể làm việc với nhau. (adapter)
- Một lớp cho phép thay đổi thuật toán được sử dụng tại thời điểm chạy bằng cách hoán đổi các đối tượng chiến lược khác nhau. (strategy)

3. Cho đoạn mã nguồn sau minh họa việc tính phí giao hàng (khi có nhiều cách tính phí giao hàng khác nhau). Hãy chọn các phát biểu đúng.

```
public class Order {
    private String shippingType;

    public Order(String shippingType) {
        this.shippingType = shippingType;
    }

    public double calculateFee(double weight) {
        if (shippingType.equals("STANDARD")) {
            return weight * 5;
        } else if (shippingType.equals("EXPRESS")) {
            return weight * 10;
        }
        return 0;
    }
}
```

- ☐ Đoạn mã vi phạm nguyên lý O (Open Closed Principle) vì mỗi khi thêm loại phí vận chuyển mới, phương thức calculateFee phải bị sửa đổi.
- ☐ Việc sử dụng chuỗi "STANDARD" và "EXPRESS" làm điều kiện rẽ nhánh khiến lớp Order phụ thuộc trực tiếp vào các chiến lược tính phí cụ thể. Đây là biểu hiện của stamp coupling.
- ☐ Thiết kế này đã áp dụng Strategy Pattern vì các thuật toán tính phí được tách thành các nhánh riêng biệt.
- ☐ Đoạn mã phù hợp để refactor bằng Strategy Pattern, bằng cách trích xuất logic tính phí thành các lớp chiến lược riêng và dùng composition thay cho điều kiện if-else.
- ☐ Cách thiết kế này là ví dụ điển hình của Template Method Pattern vì cấu trúc thuật toán đã được cố định.

4. Phát biểu nào sau đây là đúng về Strategy Pattern?

- ☐ Strategy Pattern sử dụng kế thừa để tạo ra nhiều lớp con, mỗi lớp cài đặt một biến thể thuật toán khác nhau.
- ☐ Strategy Pattern cho phép thay đổi hành vi của đối tượng tại thời điểm chạy bằng cách kết hợp (composition/aggregation/association) với một đối tượng chiến lược, thay vì mở rộng lớp bằng kế thừa. (chỉ thay đổi hành vi thay vì kế thừa cả lớp)
- ☐ Strategy Pattern buộc các thuật toán phải được cài đặt trực tiếp bên trong lớp client để giảm số lượng class trong hệ thống.
- ☐ Strategy Pattern làm tăng mức độ phụ thuộc giữa lớp client và các lớp concrete của thuật toán.
- ☐ Strategy Pattern chỉ phù hợp khi các thuật toán không bao giờ thay đổi trong quá trình thực thi chương trình.

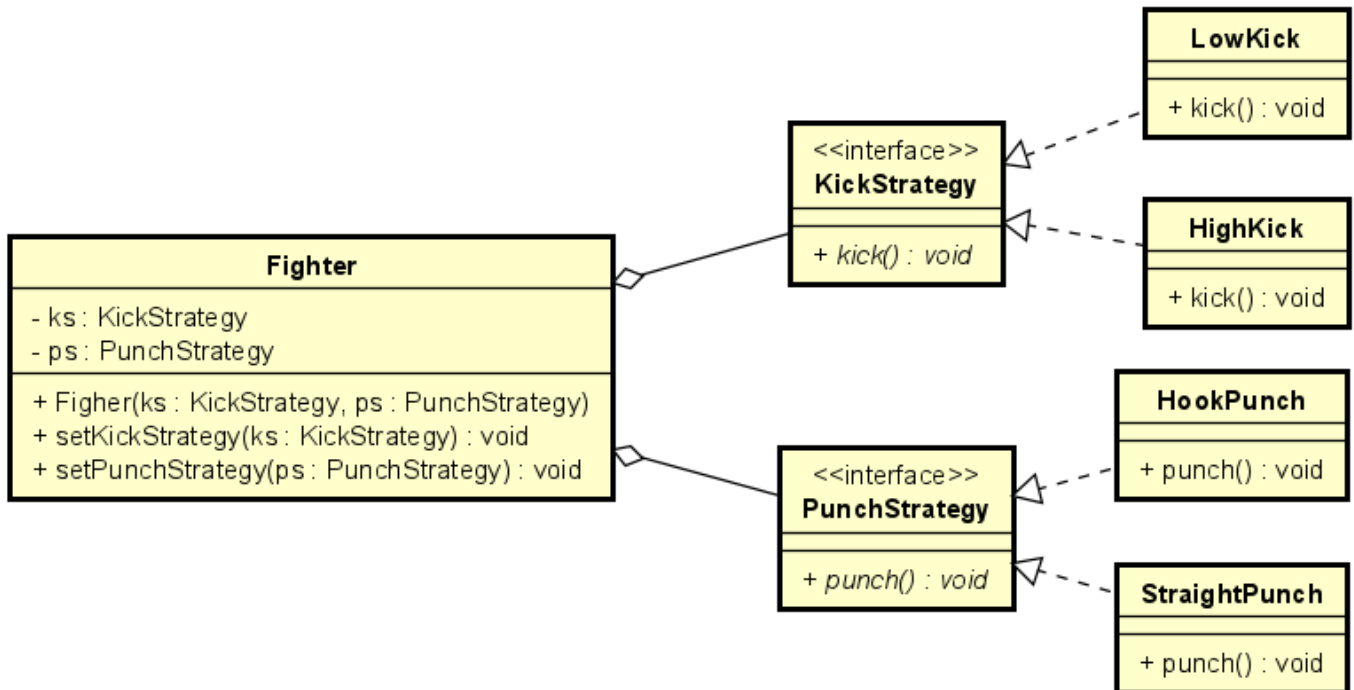
5. Factory Method Pattern thường giúp tuân thủ nguyên lý nào trong bộ S.O.L.I.D? (chọn 2 đáp án đúng nhất)

- ☐ Single Responsibility Principle (SRP)
- ☐ Open Closed Principle (OCP)
- ☐ Dependency Inversion Principle (DIP) (thằng to ko phụ thuộc phương thức cụ thể của thằng nhỏ)
- ☐ Liskov Substitution Principle (LSP) (lớp con thay thế cho lớp cha)
- ☐ Interface Segregation Principle (ISP)

6. Tình huống nào sau đây gợi nhớ đến cách sử dụng của Observer Pattern?

- ☐ Cho phép một đối tượng tự động thông báo cho nhiều đối tượng khác khi trạng thái của nó thay đổi.
- ☐ Cho phép các lớp con định nghĩa một bộ khung thuật toán nhưng hoãn việc triển khai một số bước cho lớp con.
- ☐ Cho phép hai interface không tương thích làm việc với nhau thông qua một lớp trung gian.
- ☐ Đảm bảo rằng chỉ có một thể hiện duy nhất của một lớp tồn tại trong toàn bộ chương trình.
- ☐ Giúp tạo ra các đối tượng mới mà không cần biết trước lớp cụ thể của chúng.

7. Cho thiết kế sau. Chọn các phát biểu đúng.



Strategy pattern thể hiện ở interface chứ ko phải ở implement

Giả sử có thêm 1 class Chunli thì kế thừa trực tiếp từ Fighter sang

- ☐ Thiết kế này áp dụng Strategy Pattern, trong đó các hành vi đá và đấm được đóng gói thành các strategy độc lập.
- ☐ Có thể thay đổi kiểu đá hoặc đấm của Fighter tại thời điểm chạy thông qua các phương thức setKickStrategy và setPunchStrategy mà không cần tạo đối tượng mới.
- ☐ Thiết kế này là một ví dụ của Template Method Pattern vì Fighter định nghĩa sẵn khung thuật toán chiến đấu.
- ☐ Lớp Fighter sử dụng composition với KickStrategy và PunchStrategy, qua đó ưu tiên composition over inheritance.

8. Một lớp PaymentProcessor trong hệ thống thương mại điện tử có thể xử lý nhiều phương thức thanh toán khác nhau như thẻ tín dụng, ví điện tử hoặc chuyển khoản. Hệ thống muốn có thể thay đổi phương thức thanh toán dễ dàng mà không sửa lớp PaymentProcessor. Phát biểu nào sau đây phù hợp về cách thiết kế có thể đáp ứng yêu cầu của hệ thống?

- ☐ PaymentProcessor chỉ giữ tham chiếu đến một interface/abstract class của thuật toán thanh toán (ví dụ: PaymentStrategy).
- ☐ Mỗi thuật toán thanh toán phải được triển khai trực tiếp trong lớp PaymentProcessor để đảm bảo hiệu suất.
- ☐ PaymentProcessor cần biết chi tiết nội dung/thuật toán của từng phương thức thanh toán để đảm bảo tính nhất quán.
- ☐ Có thể áp dụng Observer Pattern để đáp ứng vấn đề thay đổi phương thức thanh toán dễ dàng.
- ☐ Đây là một tình huống kinh điển để có thể áp dụng Strategy Pattern.